ARCHITECTURE DOCUMENT

Mesh-to-Parametric

Reverse Engineering Pipeline for Prismatic Parts

Onshape FeatureScript API Integration

Version 1.0 • March 2026

DRAFT — For Review

Table of Contents

1. Executive Summary

This document defines the architecture for a Python-based pipeline that converts triangulated mesh files (STL/OBJ/PLY) of prismatic mechanical parts into fully parametric CAD models with a reconstructed feature tree, delivered directly into an Onshape Part Studio via the Onshape REST API and FeatureScript.

The target part family includes brackets, mounts, plates, and similar prismatic components characterized by planar faces, cylindrical holes, pockets, bosses, fillets, and chamfers. The system does not target freeform or organic geometry.

1.1 Goals

- Accept STL, OBJ, or PLY mesh input of a single prismatic part.
- Segment the mesh into analytic surface regions (planes, cylinders, cones, spheres).
- Recognize manufacturing features: extrusions, pockets, through-holes, blind holes, fillets, chamfers.
- Reconstruct a plausible feature tree ordered by manufacturing logic.
- Push the parametric model to Onshape as editable FeatureScript features via the REST API.
- Produce a fully editable Part Studio — not a dumb imported body.

1.2 Non-Goals (v1)

- Freeform / NURBS surface reconstruction.
- Sheet metal feature recognition (bends, flanges).
- Multi-body or assembly reconstruction.
- Real-time or interactive mesh editing.

2. System Overview

The system is a six-stage pipeline. Each stage is an independent Python module with a well-defined interface, enabling unit testing, stage-level caching, and future parallelization.

#	Stage	Responsibility	Key Libraries
1	Mesh Ingestion & Cleanup	Load mesh, repair normals, remove degenerate faces, optional remeshing	trimesh, pymeshfix, numpy
2	Surface Segmentation	Cluster mesh faces into regions belonging to a single analytic surface type	trimesh, scipy, numpy, scikit-learn
3	Primitive Fitting	Fit exact analytic surfaces (plane, cylinder, cone, sphere) to each segment via RANSAC and least-squares	numpy, scipy.optimize, custom RANSAC
4	Topology & Edge Classification	Build adjacency graph, detect sharp edges, classify fillet/chamfer transitions	networkx, trimesh, numpy
5	Feature Recognition	Group primitives into semantic features (hole, pocket, boss, slot, fillet, chamfer) using rule-based pattern matching	networkx, custom rules engine
6	Onshape Export	Order features into a tree, emit Onshape API calls to create Part Studio with sketches, extrusions, holes, and fillets	requests, onshape-client, FeatureScript

3. Detailed Pipeline Design

3.1 Stage 1: Mesh Ingestion & Cleanup

Input: File path to STL (binary/ASCII), OBJ, or PLY mesh.

Output: Cleaned trimesh.Trimesh object with consistent winding, no degenerate triangles, and watertight manifold topology.

Processing Steps

1. Load mesh via trimesh.load(). Auto-detect format from extension.
2. Remove degenerate faces (area < epsilon) and duplicate vertices (merge within tolerance).
3. Fix normals: ensure consistent winding via trimesh.repair.fix_winding(). Orient outward via ray-test or volume sign.
4. Fill small holes (< N boundary edges) using trimesh.repair.fill_holes().
5. Optional: isotropic remeshing via PyMeshFix to improve triangle quality for downstream segmentation. Target edge length derived from bounding box diagonal / 200.
6. Validate: assert mesh.is_watertight and mesh.is_volume.

Configuration

- merge_tolerance: float = 1e-8 (vertex merge distance)
- min_face_area: float = 1e-10 (degenerate face threshold)
- remesh: bool = True
- target_edge_ratio: float = 0.005 (edge length as fraction of bounding box diagonal)

3.2 Stage 2: Surface Segmentation

Input: Cleaned Trimesh object.

Output: List of FaceRegion objects, each containing face indices and a preliminary surface type label.

Algorithm: Region-Growing with Normal Clustering

7. Compute per-face normals and face-face adjacency graph from the mesh.
8. Seed selection: pick the face with the most neighbors not yet assigned.
9. Region growing: from each seed, flood-fill to adjacent faces where the angle between normals is below a threshold (e.g. 15° for planar regions). Use an adaptive threshold: tighter for planes, looser for curved surfaces.
10. Post-merge: merge small regions (< 10 faces) into the neighboring region with the most similar average normal.
11. Curvature-based splitting: for each region, compute discrete curvature. If curvature variance is high, re-segment using k-means on (normal, curvature) feature vectors.

12. Label each region with a preliminary type based on curvature statistics: near-zero mean curvature → PLANE, constant positive curvature in one direction → CYLINDER, linearly varying curvature → CONE, constant curvature in both directions → SPHERE.

Data Structure: FaceRegion

- face_indices: np.ndarray of int
- surface_type: enum {PLANE, CYLINDER, CONE, SPHERE, TORUS, UNKNOWN}
- vertices: np.ndarray (Nx3) — the mesh vertices belonging to this region
- normals: np.ndarray (Nx3) — per-face normals
- boundary_edges: list of (v1, v2) tuples

3.3 Stage 3: Primitive Fitting

Input: List of FaceRegion objects.

Output: List of FittedSurface objects with exact analytic parameters and fitting error metrics.

Fitting Strategy per Surface Type

Type	Fitting Method	Parameters
Plane	SVD on point cloud: centroid = mean(points), normal = smallest singular vector of centered points	point: (x,y,z), normal: (nx,ny,nz)
Cylinder	RANSAC: sample 2 points + normals to estimate axis, then least-squares refinement of axis direction, point-on-axis, and radius	axis_point, axis_dir, radius
Cone	RANSAC: sample 3 points + normals, compute apex and half-angle, then Levenberg-Marquardt refinement	apex, axis_dir, half_angle
Sphere	Algebraic sphere fit (Pratt method) followed by geometric least-squares refinement	center: (x,y,z), radius

Validation

- Compute RMS residual (point-to-surface distance) for each fit.
- If RMS > tolerance (default 0.5% of bounding box diagonal), attempt alternative surface types.
- Snap parameters to "round" values: e.g. radius 4.998 → 5.000 if within snap tolerance.
- Detect axis alignments: if a cylinder axis is within 2° of a principal axis (X/Y/Z), snap to exact alignment.

3.4 Stage 4: Topology & Edge Classification

Input: List of FittedSurface objects + original mesh.

Output: SurfaceGraph: a networkx graph where nodes are fitted surfaces and edges are classified boundary transitions.

Algorithm

13. Build adjacency from mesh face regions: two surfaces are adjacent if they share boundary edges.
14. For each adjacency edge, classify the transition: SHARP (dihedral angle > 160° or < 20°), FILLET (bounded by a small cylindrical or toroidal region with tangent continuity to both neighbors), CHAMFER (bounded by a narrow planar strip at ~45° to both neighbors), or SMOOTH (tangent-continuous, same surface continues).
15. Compute intersection curves between adjacent analytic surfaces (e.g. plane-plane = line, plane-cylinder = ellipse). Store as parametric edge curves.
16. Build the SurfaceGraph with edge attributes: transition_type, intersection_curve, shared_edge_length, dihedral_angle.

Edge Classification Heuristics

- Fillet detection: region is cylindrical/toroidal, width < 20% of adjacent face extent, tangent to both neighbors within 5°.
- Chamfer detection: region is planar, narrow strip, angle to both neighbors is ~45° (±15°).
- Sharp edge: direct adjacency between two surfaces with dihedral angle outside tangent-continuity range.

3.5 Stage 5: Feature Recognition

Input: SurfaceGraph with classified edges + FittedSurfaces.

Output: Ordered list of Feature objects forming a feature tree.

Feature Definitions

Feature	Recognition Rule	Onshape Mapping
Base Extrusion	Largest planar face + parallel opposing face + connecting perpendicular walls. Forms the bounding envelope of the part.	Sketch on plane + Extrude (Solid, New)
Pocket	A concavity: group of inward-facing planes/cylinders fully enclosed by the base extrusion's faces. Floor plane parallel to a base face.	Sketch on face + Extrude (Remove)
Through Hole	Cylinder whose axis passes fully through the part, bounded by two planar faces.	Sketch (circle) + Extrude (Remove, Through All)

Blind Hole	Cylinder open on one face, closed by a planar floor on the other end.	Sketch (circle) + Extrude (Remove, Blind, depth)
Boss	Outward protrusion: small group of outward-facing planes on top of a larger planar face.	Sketch on face + Extrude (Solid, Add)
Slot	Elongated pocket: two parallel planes connected by two half-cylinders at ends.	Sketch (stadium shape) + Extrude (Remove)
Fillet	Cylindrical/toroidal surface tangent to two adjacent faces (see Stage 4 classification).	Fillet feature on edge(s), radius parameter
Chamfer	Narrow planar strip between two faces at approximately equal angles (see Stage 4 classification).	Chamfer feature on edge(s), distance parameter

Feature Tree Ordering

17. Base extrusion is always Feature 1. Selected as the feature whose removal would eliminate the most volume.
18. Additive features (bosses) next, ordered largest-to-smallest by volume contribution.
19. Subtractive features (pockets, holes, slots) next, ordered largest-to-smallest.
20. Fillets and chamfers last — they depend on edges created by prior features.
21. Dependency resolution: if Feature B references a face created by Feature A, A must precede B. Use topological sort on the dependency graph.

3.6 Stage 6: Onshape Export

Input: Ordered list of Feature objects.

Output: A fully parametric Onshape Part Studio with an editable feature tree.

Onshape API Integration Strategy

The pipeline uses the Onshape REST API (v6) to programmatically build the Part Studio. The approach is to create each feature sequentially via the Add Feature endpoint, which accepts FeatureScript-encoded feature definitions.

API Workflow

22. Authentication: OAuth2 or API Key-based authentication via the onshape-client Python library.
23. Create a new Document and Part Studio, or target an existing one.
24. For each feature in the ordered tree, call POST /api/v6/partstudios/d/{did}/w/{wid}/e/{eid}/features with the FeatureScript JSON payload.
25. After each feature, query the Part Studio to retrieve the created face/edge IDs (transient IDs) needed for subsequent features that reference those faces.
26. For fillets/chamfers, use the edge query API to find edges by adjacency to known faces.

FeatureScript Payload Examples

Each feature is represented as a JSON body conforming to the Onshape BT Feature Definition schema. Key feature types and their API mapping:

- Sketch + Extrude: Create a BTMSketch with sketch entities (lines, arcs, circles) on a specified plane, then a BTMFeature for fOpExtrude with depth, direction, and operation type (NEW, ADD, REMOVE).
- Hole: Use fOpExtrude with a circle sketch, or fHole for standard hole patterns with countersink/counterbore options.
- Fillet: fOpFillet referencing edge queries, with radius parameter.
- Chamfer: fOpChamfer referencing edge queries, with distance parameter.

Face/Edge Resolution

Onshape uses transient queries (deterministic IDs computed from the feature tree state) to reference geometry. The pipeline must:

- After creating the base extrusion, enumerate all resulting faces and map them back to the fitted surfaces from Stage 3 by comparing face normals and positions.
- For each subsequent feature, construct a transient query that selects the correct face. For example: "the planar face of the first extrude whose normal is (0, 0, 1)" maps to a qCreatedBy + qGeometry filter.
- Cache the mapping of (FittedSurface ID → Onshape transient query) and update it after each feature addition.

4. Data Model

The pipeline uses a set of dataclasses passed between stages. All are serializable to JSON for caching and debugging.

Class	Key Fields	Used By
CleanedMesh	vertices, faces, normals, is_watertight, bbox	Stage 1 → Stage 2
FaceRegion	face_indices, surface_type, vertices, normals, boundary_edges	Stage 2 → Stage 3
FittedSurface	region_id, surface_type, params (dict), rms_error, snapped_params	Stage 3 → Stage 4, 5
SurfaceEdge	surface_a_id, surface_b_id, transition_type, dihedral_angle, curve_params	Stage 4 → Stage 5
Feature	type, sketch_geometry, depth, direction, operation, dependencies, onshape_query	Stage 5 → Stage 6
FeatureTree	features (ordered list), dependency_graph, metadata	Stage 5 → Stage 6

5. Project Structure

The codebase is organized as a Python package with clear separation between pipeline stages, data models, and the Onshape integration layer.

Directory Layout

- mesh2param/ — top-level package
  - mesh2param/pipeline/ — stage modules: ingest.py, segment.py, fit.py, topology.py, recognize.py, export_onshape.py
  - mesh2param/models/ — dataclasses: mesh.py, region.py, surface.py, feature.py, tree.py
  - mesh2param/onshape/ — API wrapper: client.py, auth.py, feature_builder.py, query_builder.py
  - mesh2param/utils/ — geometry helpers, RANSAC, snapping, logging
- mesh2param/cli.py — Click-based CLI entry point
- mesh2param/config.py — Pydantic settings with YAML override
- tests/ — pytest suite with fixtures per stage
- examples/ — sample STL files and expected outputs
- docs/ — Sphinx documentation

CLI Interface

The primary entry point is a CLI tool:

- mesh2param convert input.stl --onshape-doc "My Document" --onshape-workspace main
- mesh2param convert input.stl --dry-run --output feature_tree.json (export tree without pushing to Onshape)
- mesh2param inspect input.stl --stage segment --visualize (run up to a stage and visualize results)

6. Onshape API Reference

The pipeline uses the following Onshape REST API endpoints. All require authentication via OAuth2 or API keys.

Operation	Endpoint	Purpose
Create Document	POST /api/v6/documents	Create target document
Add Feature	POST /api/v6/partstudios/d/{did}/w/{wid}/e/{eid}/features	Add each feature to the tree
Get Features	GET /api/v6/partstudios/d/{did}/w/{wid}/e/{eid}/features	Read back feature IDs
Evaluate FS	POST /api/v6/partstudios/d/{did}/w/{wid}/e/{eid}/featurescript	Run FeatureScript to query faces/edges
Get Faces	GET /api/v6/partstudios/.../tessellatedfaces	Map fitted surfaces to Onshape faces

Authentication Setup

Users must create an Onshape Developer Portal application and obtain either API keys (access key + secret key) or configure OAuth2. The pipeline stores credentials in environment variables: ONSHAPE_ACCESS_KEY, ONSHAPE_SECRET_KEY, or ONSHAPE_OAUTH_TOKEN.

7. Risks, Limitations & Mitigations

Risk / Limitation	Impact	Mitigation
Ambiguous feature tree	Multiple valid trees exist for the same shape. The reconstructed tree may not match original design intent.	Use manufacturing-logic heuristics. Allow user to reorder in Onshape after export.
Noisy / low-res meshes	Segmentation errors propagate through the pipeline.	Require mesh quality checks at ingestion. Provide remeshing. Expose quality scores per feature.
Complex blends / transitions	Variable-radius fillets, G2 blends, or unusual transitions may not be recognized.	Fall back to constant-radius fillet approximation. Flag unrecognized regions for manual review.
Onshape API rate limits	Complex parts with 50+ features may hit rate limits during sequential creation.	Batch features where possible. Implement exponential backoff. Cache intermediate results.
Transient query fragility	Onshape transient queries can break if the feature tree changes unexpectedly.	Validate each feature addition before proceeding. Use robust query strategies (qCreatedBy + qGeometry).
Non-prismatic regions	Freeform surfaces present in an otherwise prismatic part will not be handled.	Detect and flag as UNKNOWN. Export as imported mesh region. Alert user.

8. Testing Strategy

8.1 Unit Tests

- Each pipeline stage has isolated tests with known input/output fixtures.
- Primitive fitting: test with synthetically generated point clouds (exact plane, cylinder, etc.) with known parameters + Gaussian noise.
- Feature recognition: test with hand-labeled surface graphs.

8.2 Integration Tests

- End-to-end: run a known STL through the full pipeline and compare the output feature tree to an expected JSON fixture.
- Onshape round-trip: push features to Onshape, export as STEP, compare bounding box and volume to the original mesh within tolerance.

8.3 Benchmark Parts

A curated set of 10–20 STL files covering the target part family:

- Simple L-bracket (2 planes, 4 holes, 2 fillets)
- Motor mount plate (rectangular, 6 holes, 2 pockets)
- T-bracket (3 planes, bosses, countersunk holes)
- Gusset plate (triangular profile, slots, fillets)
- DIN-rail mount (complex pocket, snap-fit features)

9. Implementation Roadmap

Phase	Timeline	Deliverable	Success Criteria
Phase 1	Weeks 1–3	Stages 1–3: mesh cleanup, segmentation, primitive fitting. CLI with --dry-run JSON output.	Fits planes/cylinders on 5 test STLs with RMS < 0.1% bbox.
Phase 2	Weeks 4–6	Stages 4–5: topology, edge classification, feature recognition. Outputs a feature tree JSON.	Correctly identifies holes, pockets, fillets on L-bracket and motor mount.
Phase 3	Weeks 7–10	Stage 6: Onshape API integration. Full end-to-end pipeline.	Pushes editable feature tree to Onshape for 3 benchmark parts.
Phase 4	Weeks 11–12	Hardening: error handling, logging, documentation, expanded benchmark suite.	All 10 benchmark parts convert with < 5% volume error.

10. Dependencies & Environment

10.1 Python Dependencies

- trimesh >= 4.0 — mesh loading, repair, adjacency
- numpy >= 1.24 — array operations
- scipy >= 1.11 — optimization, spatial algorithms
- scikit-learn >= 1.3 — clustering for segmentation
- networkx >= 3.1 — surface adjacency graph
- pymeshfix >= 0.16 — mesh repair and remeshing
- requests >= 2.31 — HTTP client for Onshape API
- click >= 8.1 — CLI framework
- pydantic >= 2.0 — configuration and data validation
- pytest >= 7.4 — testing

10.2 Environment

- Python 3.11+
- Onshape Professional or Enterprise account (API access required)
- Environment variables for Onshape credentials

End of Document